



Control Lab

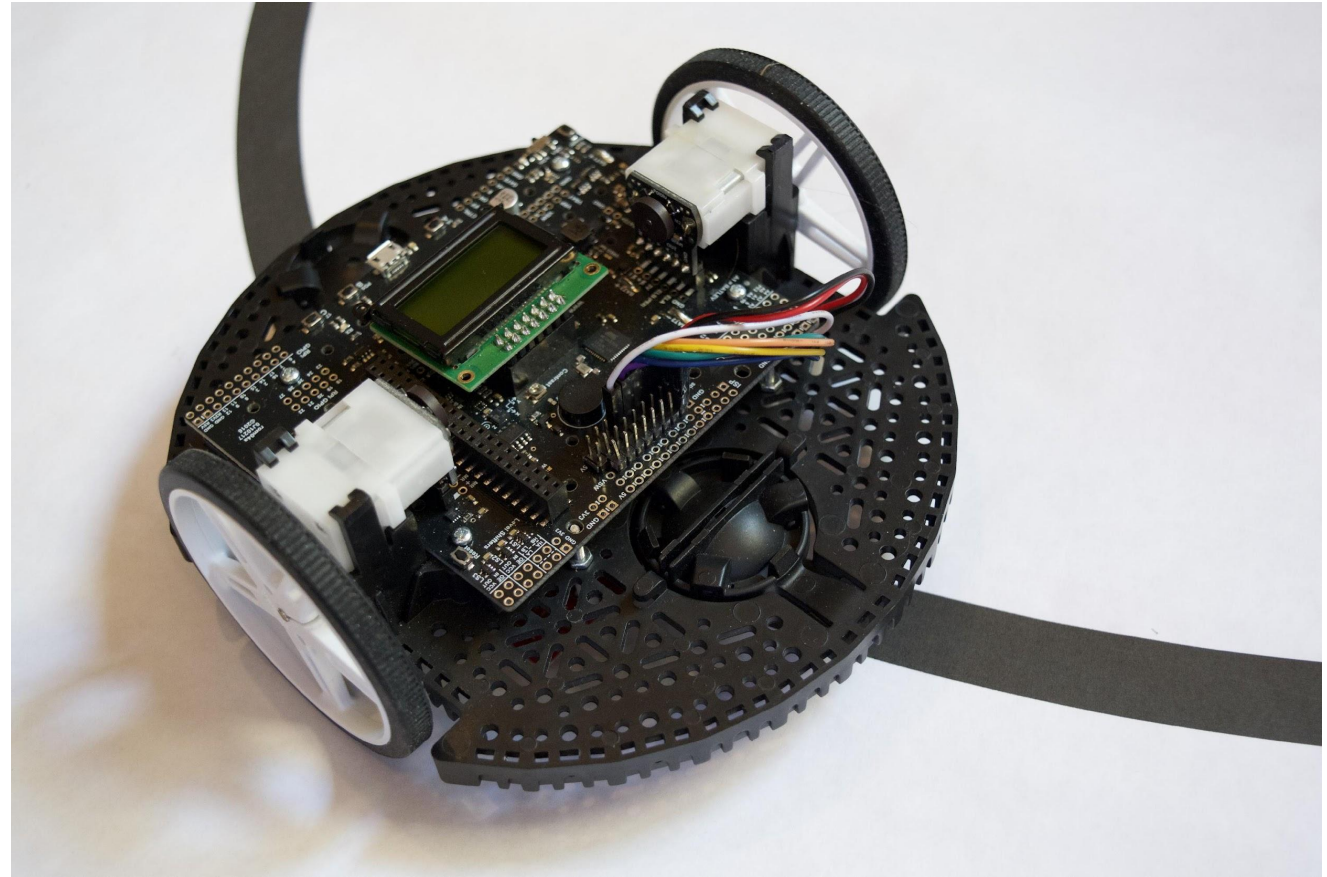
Fall 2026

Overview

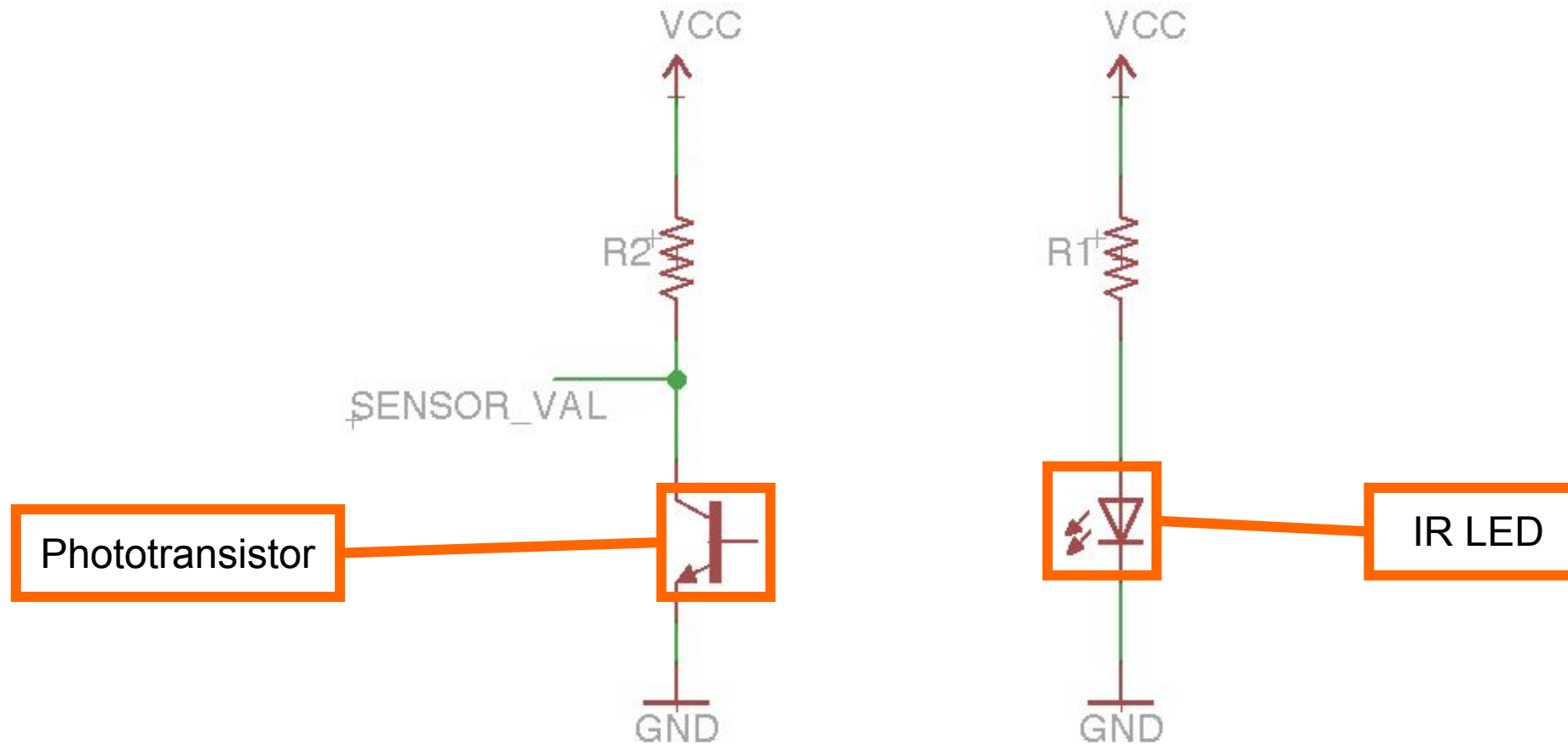
- Lab Goals
- Reflectance Sensors
- Control Intro
- Proportional Control
- Proportional/Derivative Control
- Advanced Control

Lab Challenge

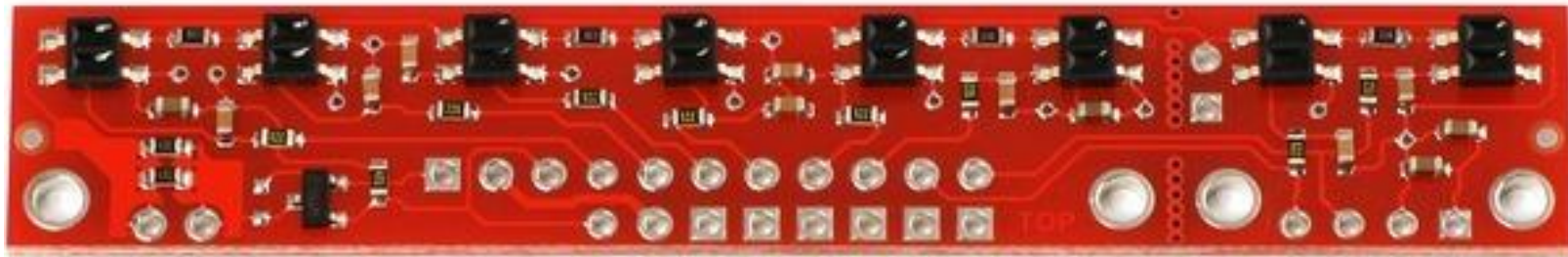
- You need to program your robot to follow our line
- The robot needs to complete 2 laps without leaving the line
- 45 sec for two laps



Reflectance Sensors



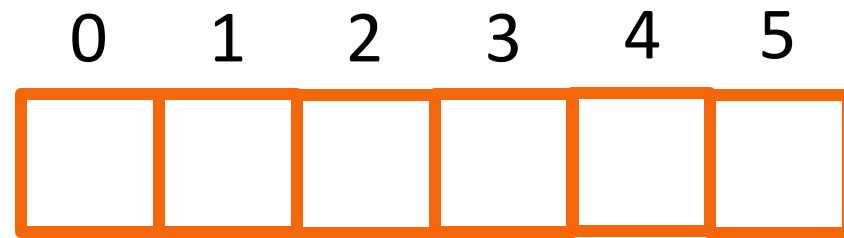
Reflectance Sensors



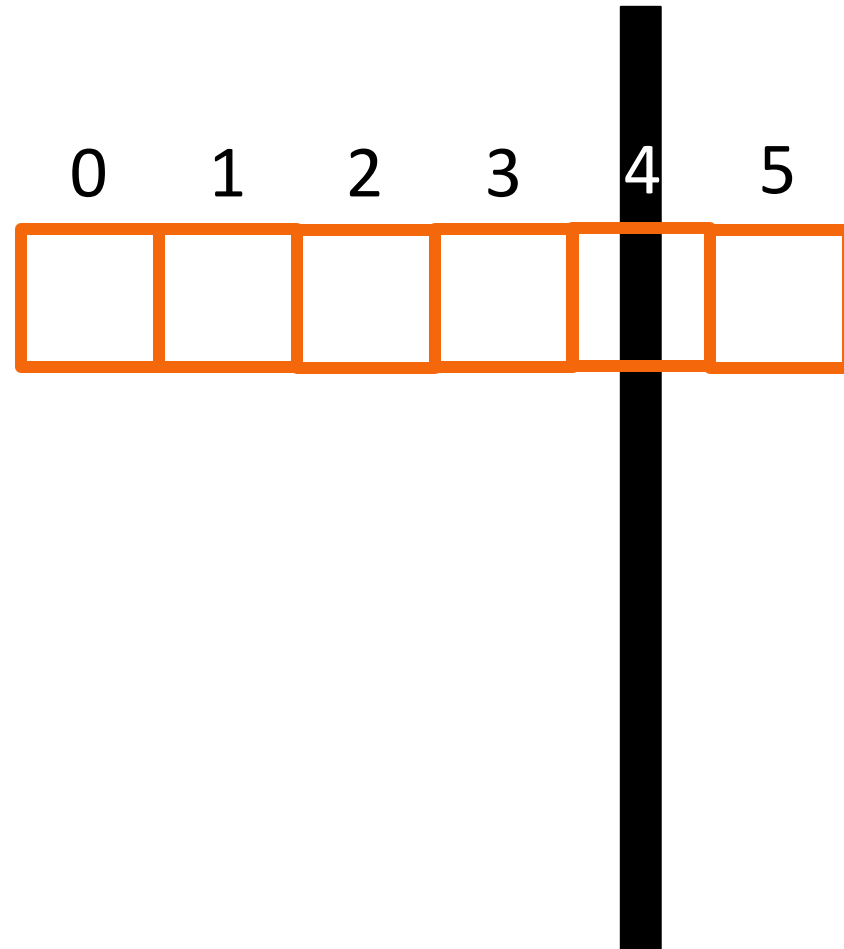
readLine function

- To figure out where the line is, use:
`qtrrc.readLine(sensors);`
- This will read all the sensors and figure out where the line is

How readLine Works

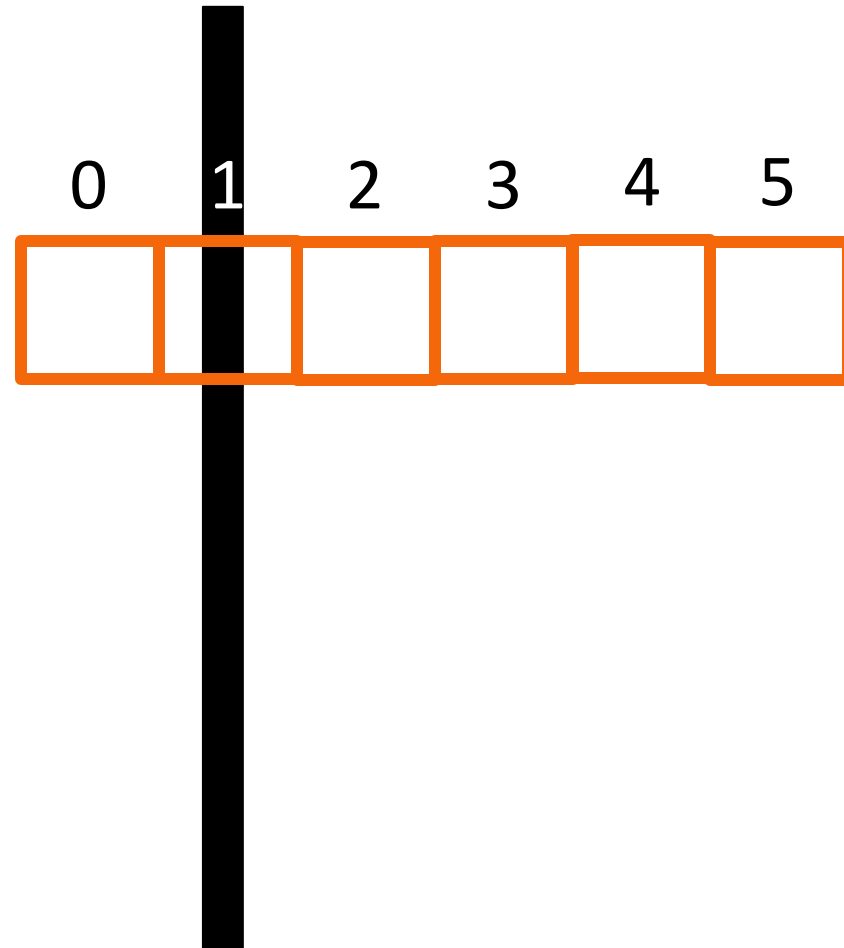


How readLine Works



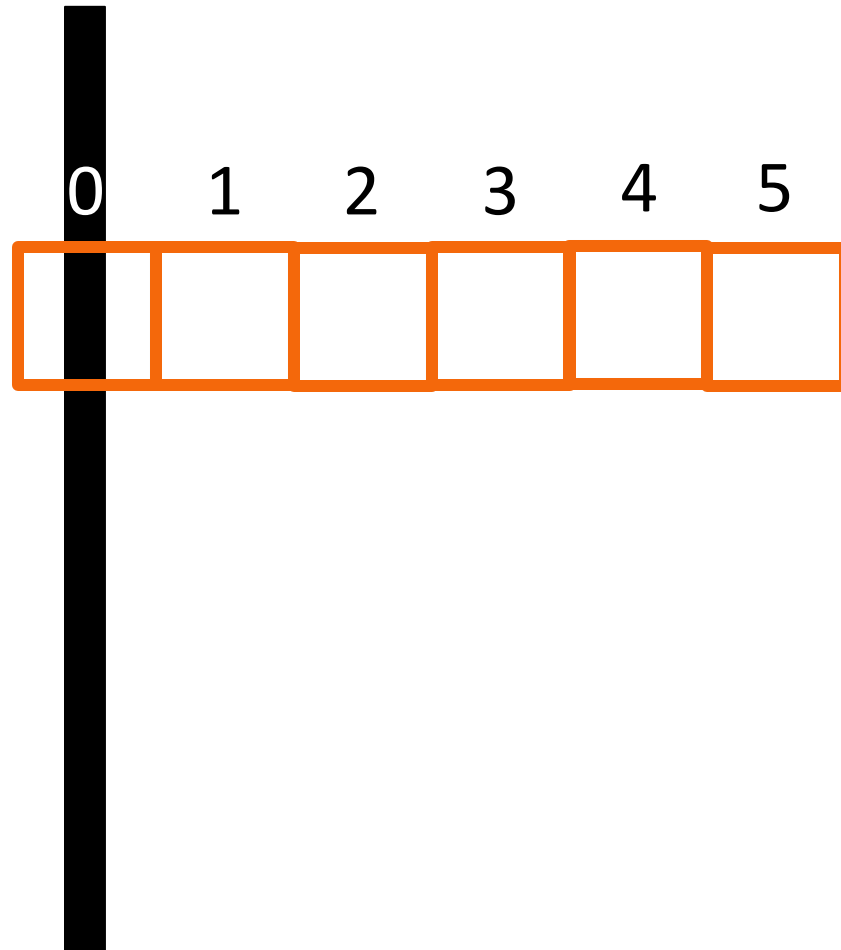
Output:
~4000

How readLine Works



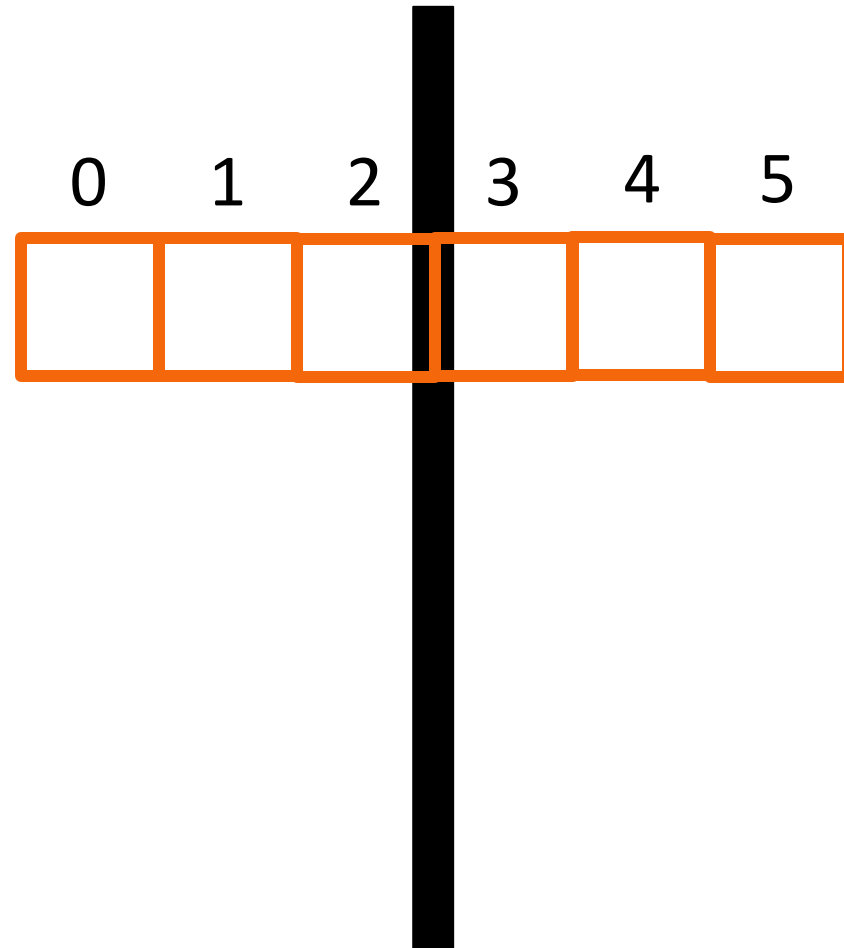
Output:
~1000

How readLine Works



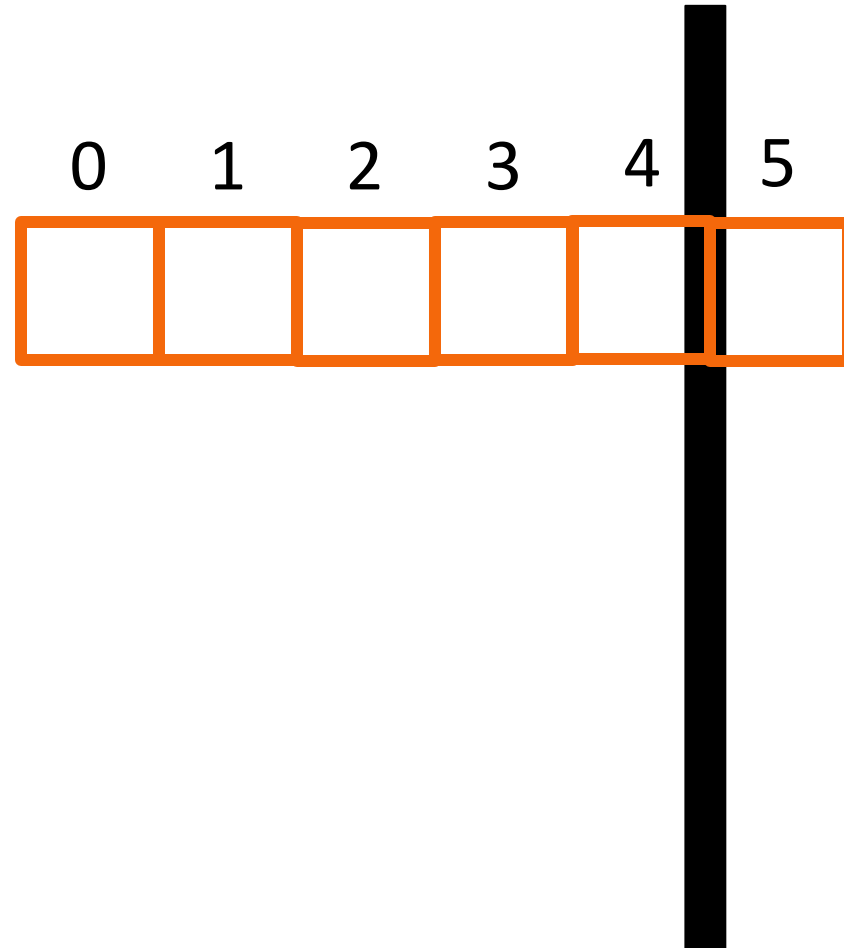
Output: ~0

How readLine Works



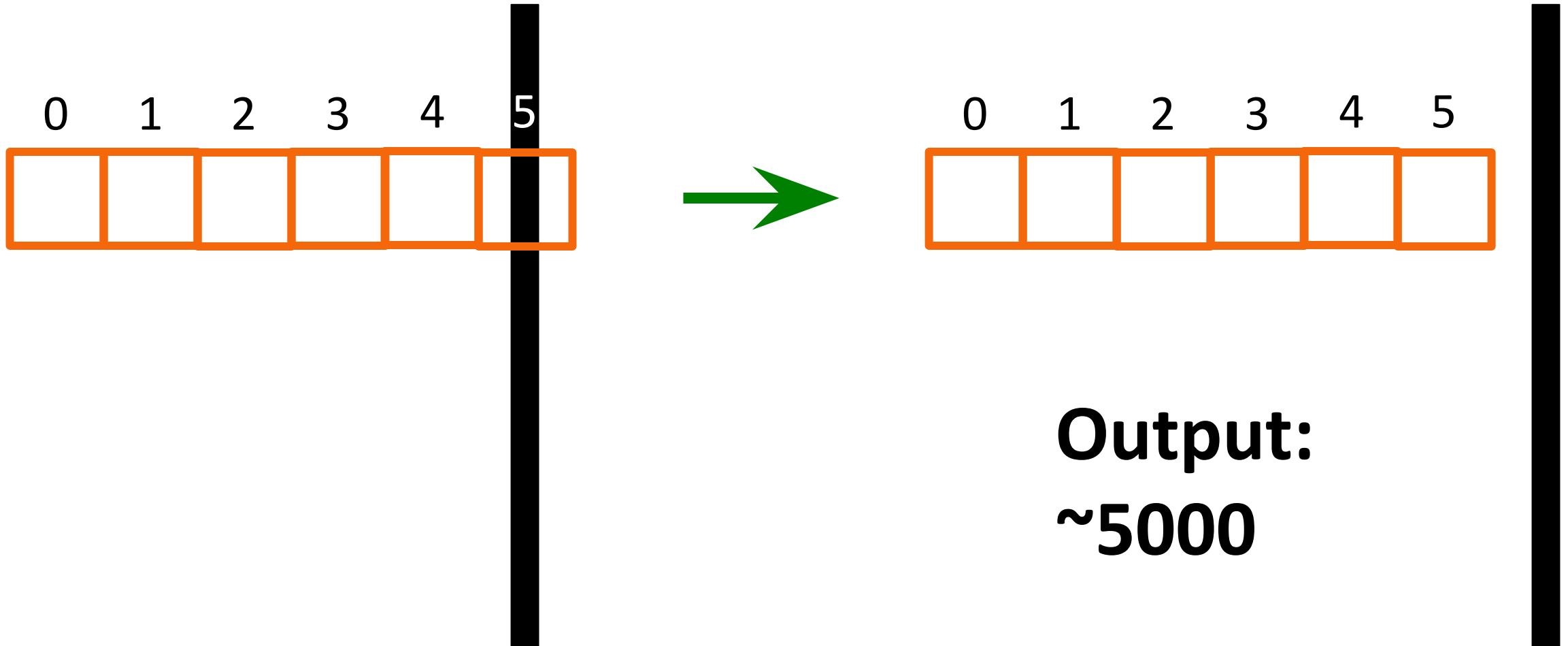
Output:
~2500

How readLine Works



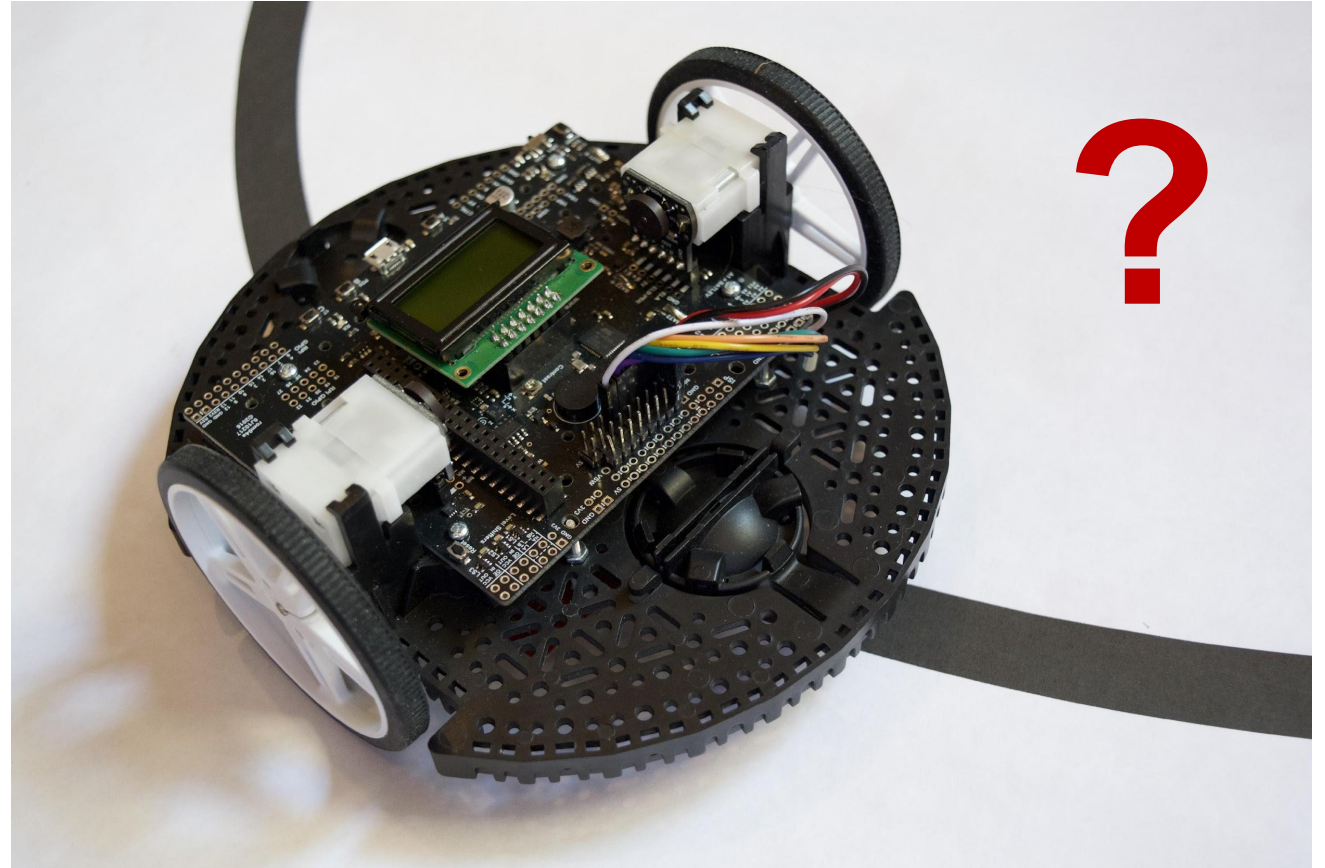
Output:
~4500

How readLine Works



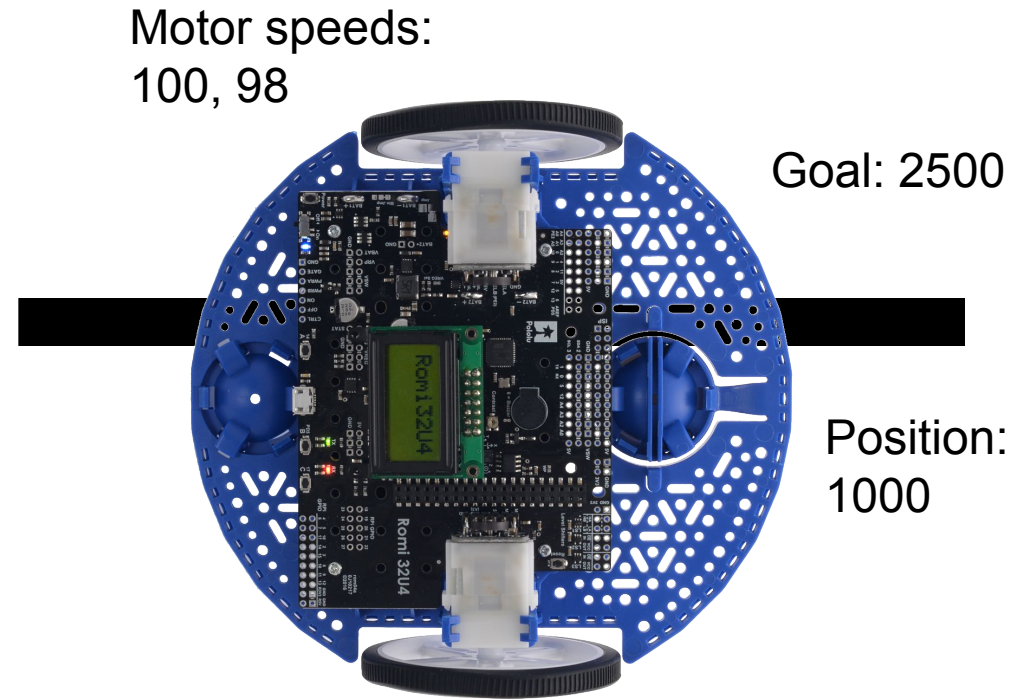
Robot Control

- We know where the line is, but how do we follow it?

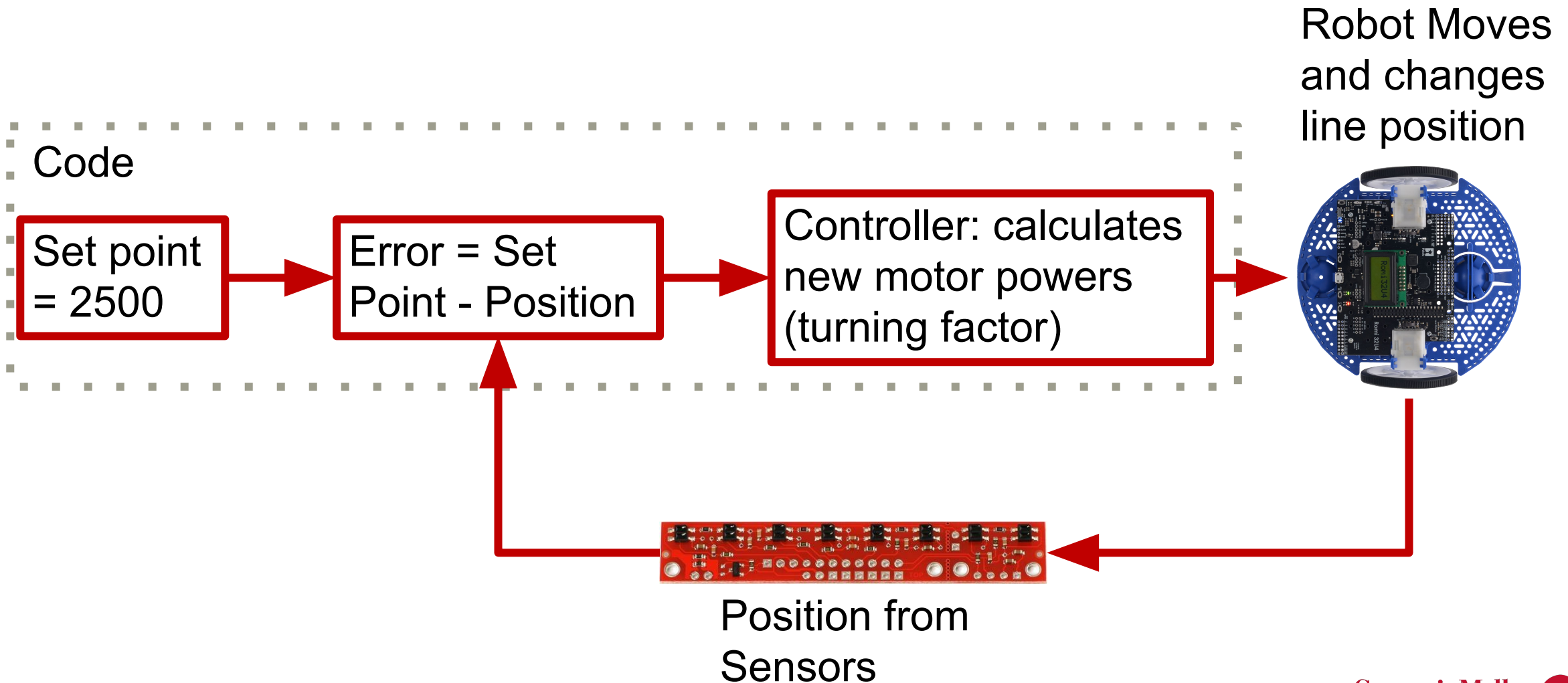


The Control Problem

- Knowns
 - Current position of line
 - Where we want the line to be
- Unknown
 - What motor values will get us to our goal



The Control System



Candy Question!

- What are some real-life problems that would require control theory and sensors to control autonomously?
 - Example: Balancing on a beam



Control Systems in the Wild

- What are similar problems in real life:
 - Maintaining temperature in a room
 - Move a robotic arm to a specific angle (and keep it there)
 - Keeping a car in its lane (Very similar to line following)
 - Landing a rocket
- Real World Controllers
 - Bang-Bang controller
 - [Flyball Governor](#)
 - PID controller

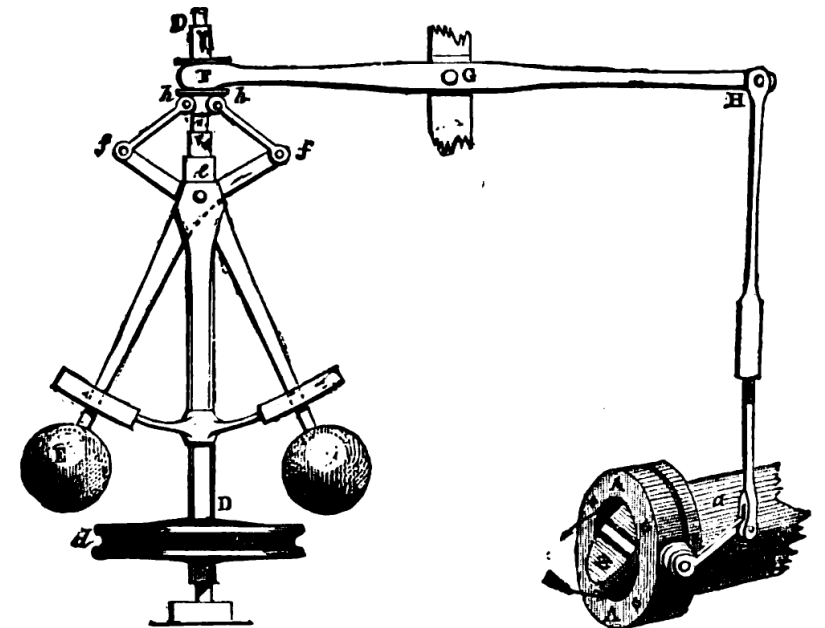
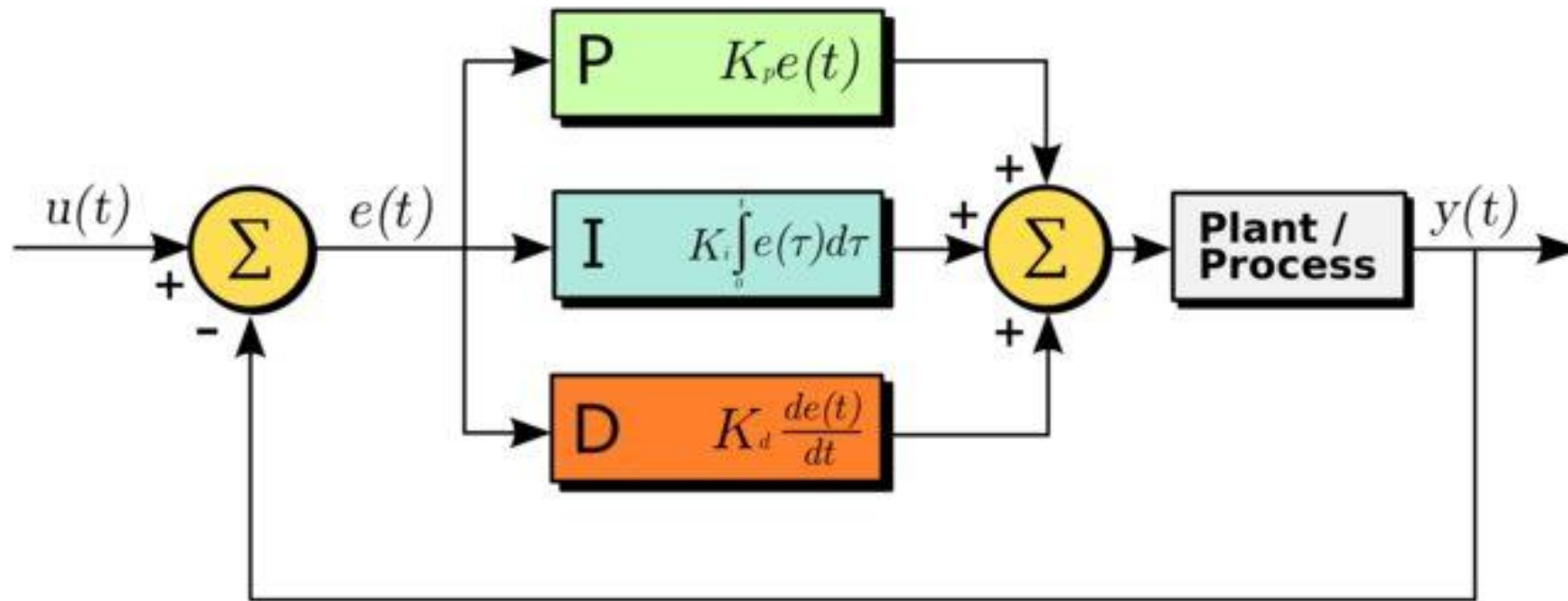


FIG. 4.—Governor and Throttle-Valve.

PID Control

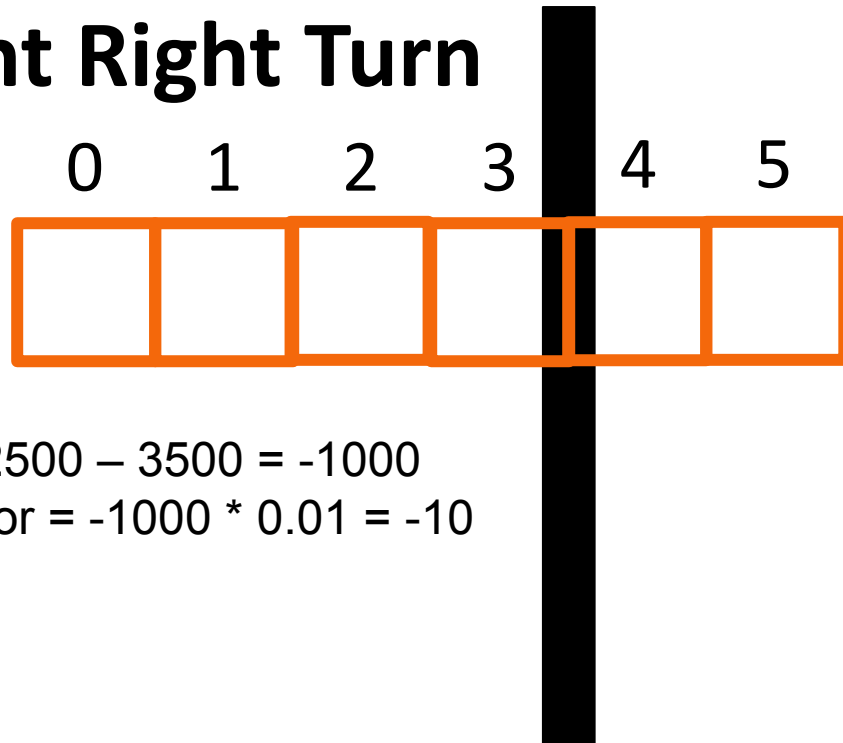
- Output = Proportional * K_p + Integral * K_i + Derivative * K_d



Proportional Control

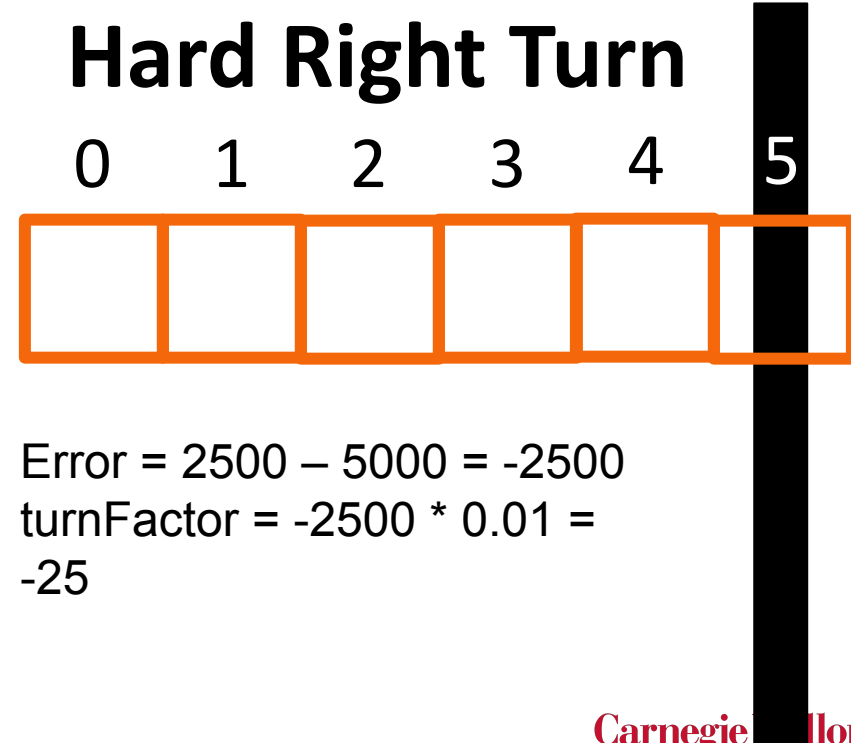
- Change how strongly you turn by how far off you are from your goal

Slight Right Turn



$$\text{Error} = 2500 - 3500 = -1000$$
$$\text{turnFactor} = -1000 * 0.01 = -10$$

Hard Right Turn



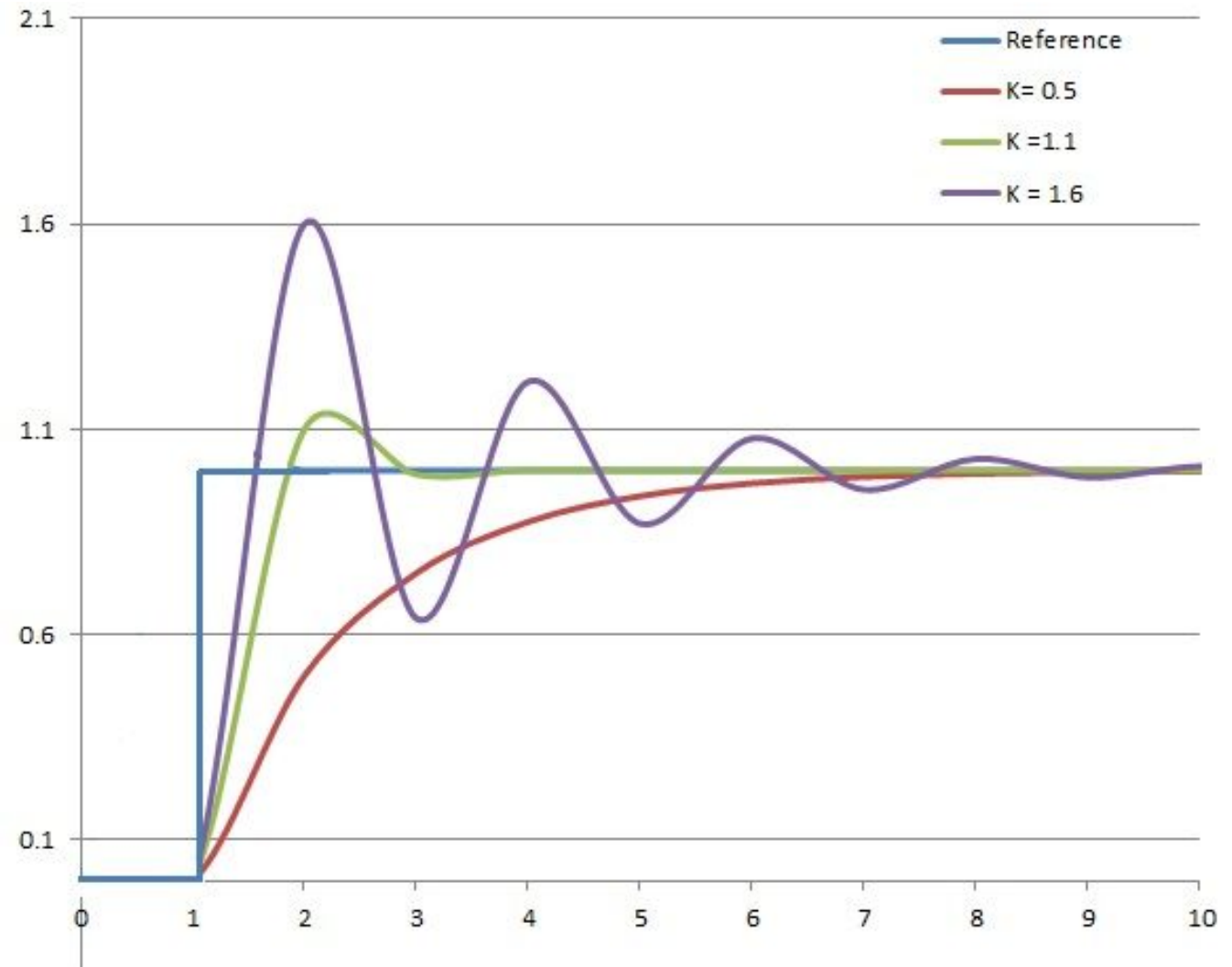
$$\text{Error} = 2500 - 5000 = -2500$$
$$\text{turnFactor} = -2500 * 0.01 = -25$$

Proportional Control

- The constant factor - K_p
- Different systems need different K_p 's based on the system's sensors and actuators as well as the environment
- Goal: Minimize time to correct an error and overshoot
 - Too small and you never reach the goal
 - Too big and you end up overshooting and then oscillate
- [Tuning proportional control](#)

Candy Question

What is the best K_p on the graph?



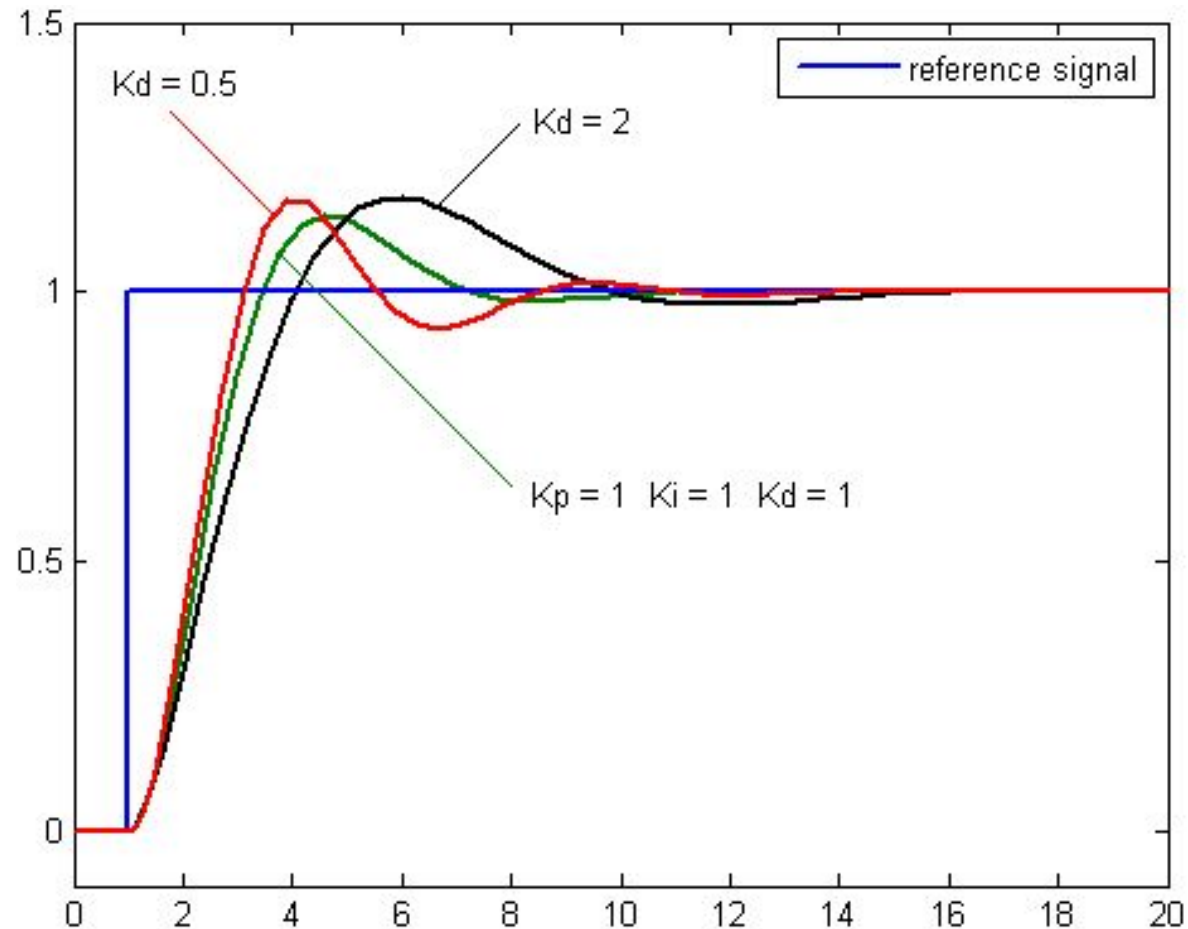
Derivative Control

- Derivative is the D in PID control
- Proportional on how fast the error is changing (the error's derivative)
- Slow down extra if approaching set point really quickly
 - This helps prevent oscillation on tight turns
- Can amplify noise, so may need to filter error
 - More on this next week

Derivative Control

- Derivative is needed to smooth out the effects of the proportional movement: it's also called a damping term.
- When approaching an end state, we want to smoothly stop
- Too big of a k_D causes the robot to slow down too much when reaching a goal.
- Too small of a k_D causes the robot to still overshoot
- Optimally start tuning k_D after k_P is tuned (increase k_P by a bit when implementing k_D)

Graph!



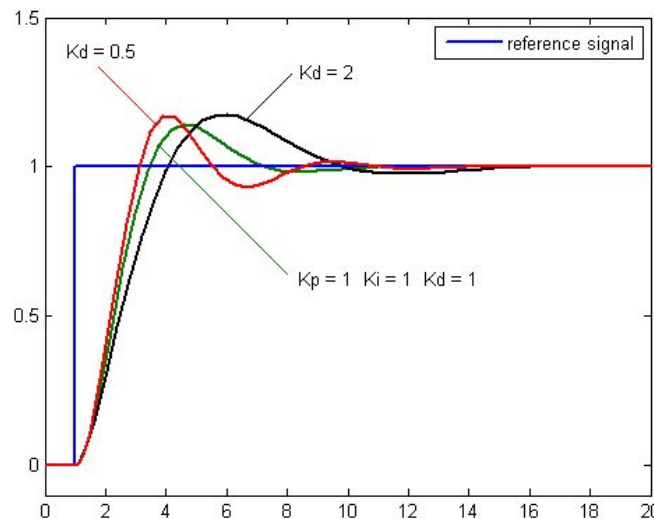
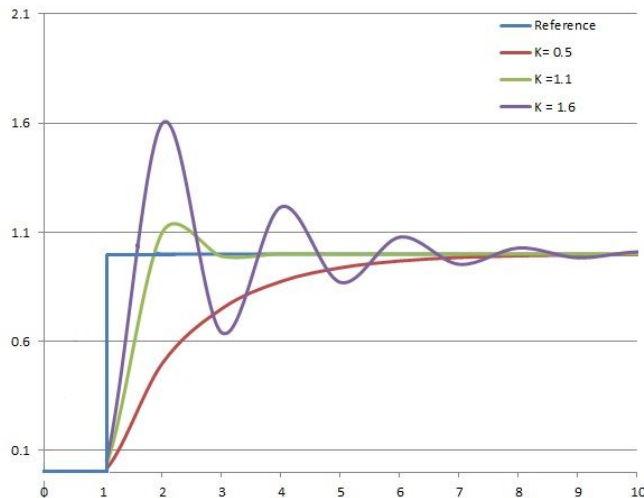
Derivative Control

- Derivative of the error is multiplied by a constant, K_d
- K_d must be tuned
- To calculate the Derivative of the error:

```
int old_err = 0;
void loop() {
    // calculate position and error
    int derivative = error - old_err;
    // calculations
    old_err = error;
}
```

Candy Question

- Why do we need to increase k_P after tuning it before introducing k_D ?
- If k_D is too big, then the robot reaches its goal too slowly. What k_P error would cause the same thing?



Answer

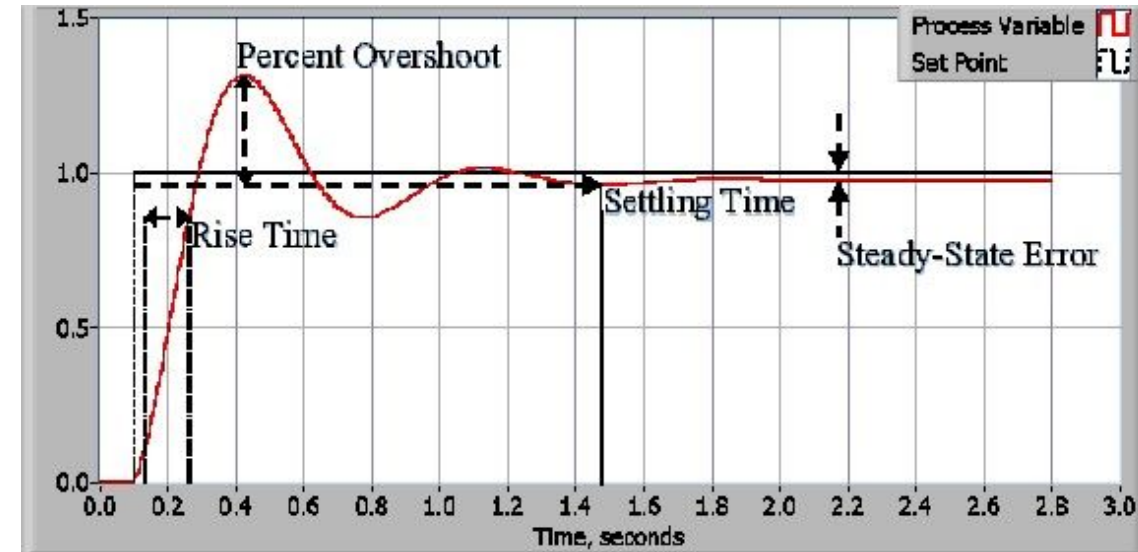
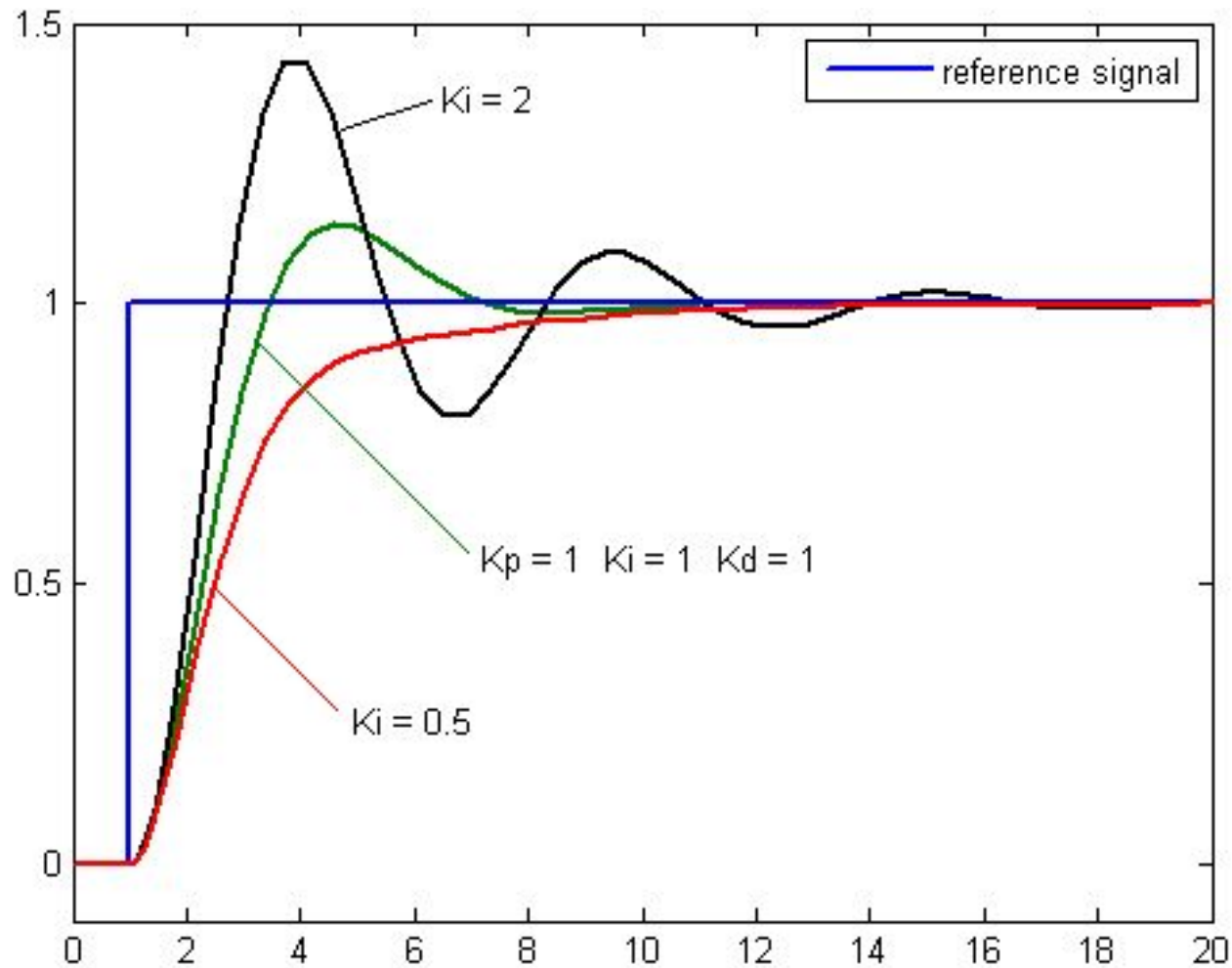
As we mentioned before, k_D will slow down the movement of the robot near the goal, which is going to influence the strength of k_P , therefore we increase k_P by a bit to compensate for this change.

If k_D is too big, the robot acts as if k_P is too small. In other words, if k_D is too big, k_P is damped too much and becomes too small.

Integral Control

- The I in PID
- The sum of the error over time (multiplied by K_i)
- Gets rid of “steady-state” error
 - We don't have this in line following
- Can be tough to implement well
 - Can cause crazy oscillations when too high, often needs other terms that go along with it (such as integral delay or integral window)
- Needed when a physical obstacle occurs that is not related to the error

Graph!



Candy Question!

- Why **don't** we need an integral term for line following?



Answer

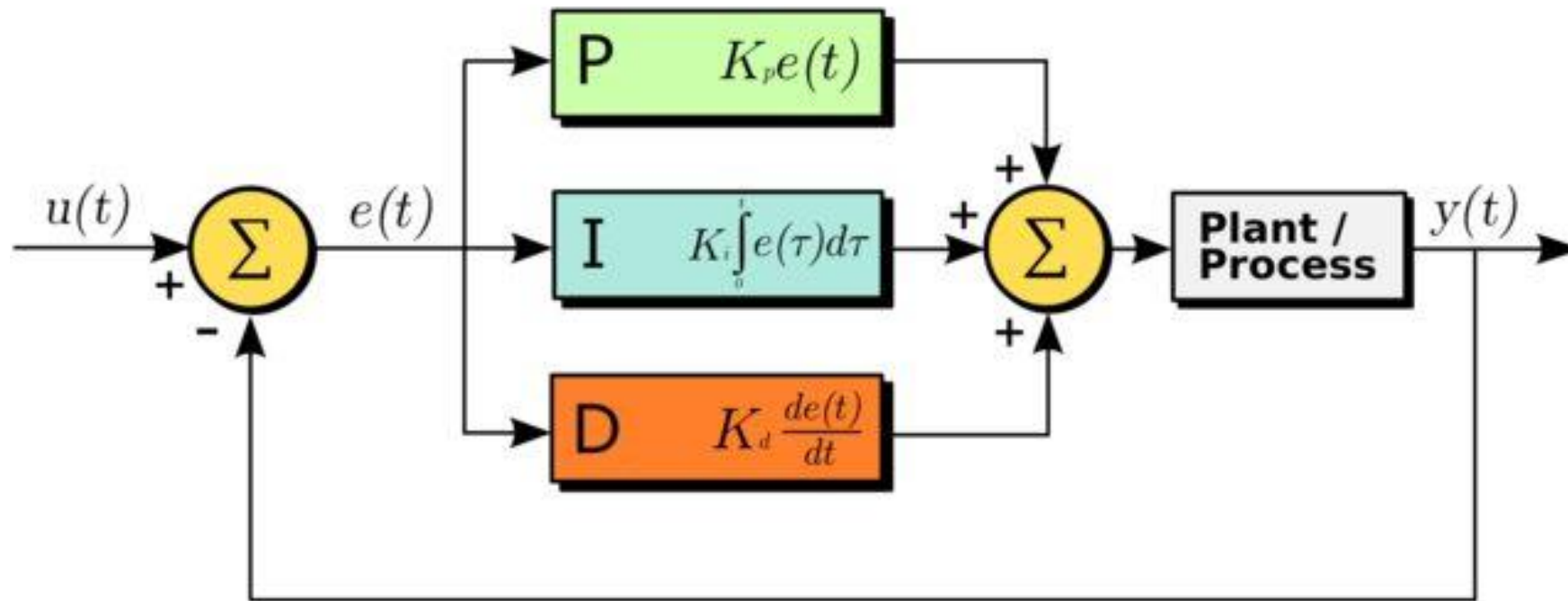
- The integral term is mostly used when the robot (say, a robotic arm) is supposed to be at an angle of X degrees but can't quite make it there every time due to factors like gravity acting on it.
- In line following, there is no final goal we are trying to reach and no steady state error. The line is constantly shifting so it would not really be useful to have an integral term.

Feed Forward

- Usually denoted as k_B
- Interchangeable with integral control
 - Easier to use, less likely to cause crazy errors.
- Constant value on the robot unrelated to the error
- In terms of a moving robot it can be considered as its base speed
- It can be arbitrary or based on the relation between input and output of an algorithm.

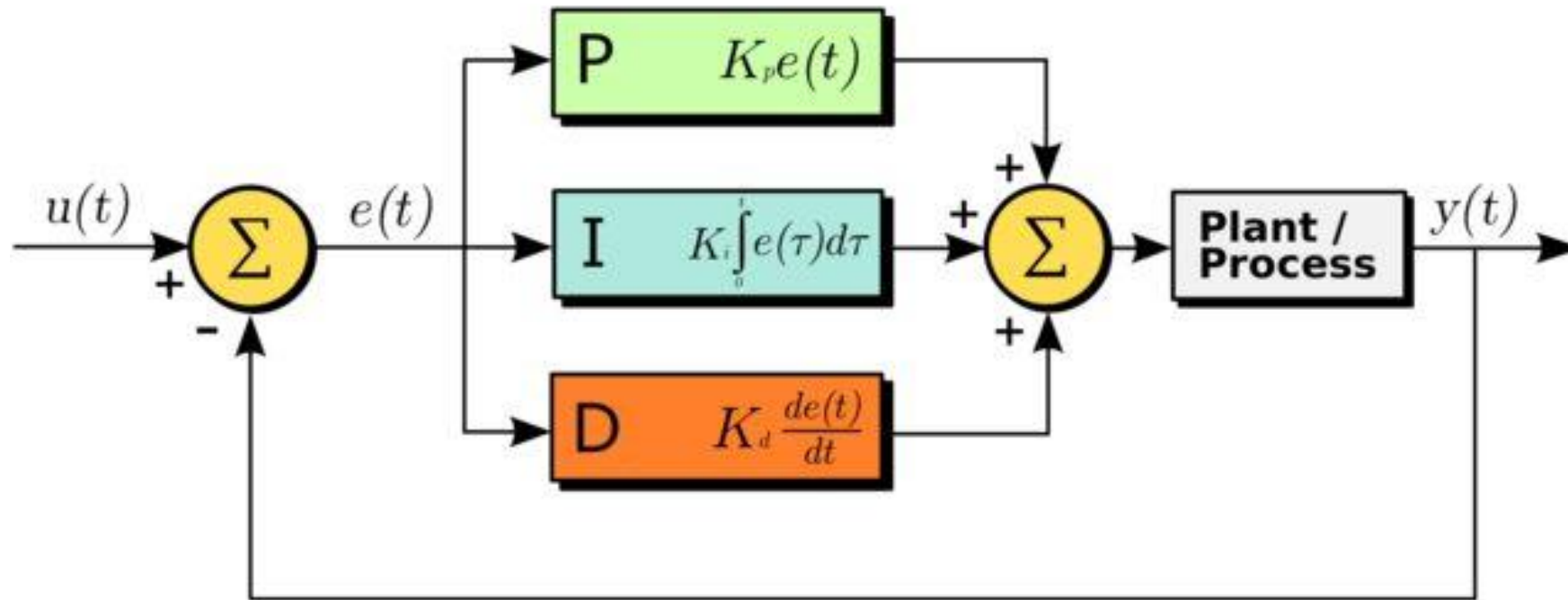
PID Control

- Output = Proportional * K_p + Integral * K_i + Derivative * K_d



PID Control

- Output = Feed Forward
+/- Proportional * K_p + Integral * K_i + Derivative * K_d



Note on Delays

- Control loops rely on repeated updating
- New outputs are calculated each loop
- You want to do this as often as possible
- Long delays = slow reaction time

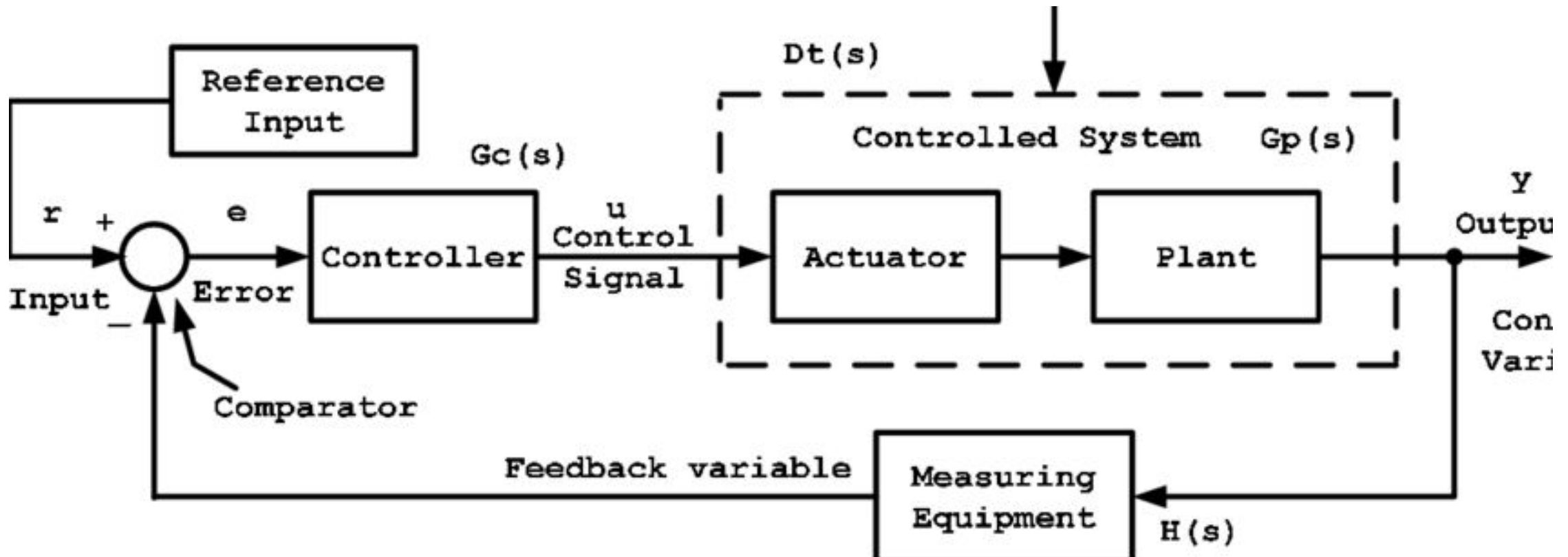


Installing the QTR Library

- [Installation Instructions](#)

Bonus Candy Question

What is the reference input, controller, and output in our system? What is the measuring equipment?



Answer:

- **Input:** Desired position for robot
- **Controller:** PID
- **Actuator:** Robot's motors
- **Plant:** Robot
- **Output:** Robot's current position
- **Feedback Variable:** Robot's current position, sensed

